AMD

Repo2Reproduce: Skill-Guided Reproduction of Research Results on AMD ROCm

Chaitanya Manem [1] **Pretam Ray** [1] **Gaurav Sarkar** [2 †]
Umang Pandey [1] **Pratik Prabhanjan Brahma** [1] **Emad Barsoum** [1]
[1]Advanced Micro Devices, Inc. (AMD) [2]IIIT Hyderabad

Abstract

Reproducing a published machine-learning result requires more than executing its released code. The target experiment and its scientific configuration must be reconstructed from evidence distributed across the paper, appendix, and repository. Because these sources may be incomplete or mutually inconsistent, their conflicts must be identified and resolved before execution. Hardware migration introduces a further challenge when CUDA-specific components must run on AMD GPUs without silently replacing accelerator-critical kernels. We present **Repo2Reproduce**, a skill-guided agentic system that transforms a paper and its official repository into an auditable reproduction on AMD Instinct™ GPUs. The system freezes a paper-grounded target, reconciles experimental settings between the paper and code repository, prepares a ROCm execution environment, and runs the experiment under deterministic completion and bounded-repair controls. Evidence-grounded finalization then compares the extracted result with the exact paper target. For CUDA-dependent paths, a porting mechanism combines source analysis, HIPIFY-assisted translation, targeted kernel rewrites, and numerical-parity checks. We also introduce **Repo2ReproduceBench**, a 22-task benchmark comprising a diverse 20-paper reproduction core and a two-task kernel-porting extension under practical single-GPU constraints. On the paired core, Repo2Reproduce improves the claim-level match rate from 70% to 85% relative to a general-purpose coding agent and reduces incomplete runs from three to zero. Both porting tasks satisfy their numerical-parity criteria, although performance agreement remains architecture-dependent. These results demonstrate that specialized reasoning skills, controlled tool use, and explicit scientific provenance make automated reproduction more faithful and auditable than repository execution alone.

1 Introduction

A research claim can be independently examined only when another researcher can recover the procedure that produced it. For modern machine-learning papers, that procedure is rarely contained in one place. The article motivates the method and reports selected results; supplementary material adds experimental details; the repository encodes several possible workflows; and datasets, checkpoints, dependency versions, and preprocessing instructions may be hosted elsewhere. These artifacts jointly describe the experiment, but none is necessarily complete on its own [1, 2, 3]. Reproduction is therefore a problem of recovering a missing experimental specification before it is a problem of running code.

Recovering that specification is difficult because the available sources can disagree. Values reported in a paper may differ from defaults in a configuration file. A README command may launch a demonstration rather than the experiment behind a headline result. Multiple scripts may use similar names while implementing training, evaluation, ablation, or debugging variants. Even after the relevant path is found, dependency drift, unavailable artifacts, implicit preprocessing, and machine-specific assumptions can prevent execution [2, 3]. Resolving these issues requires preserving the relationship between a reported claim and the exact computation used to examine it.

Reproduction becomes harder when the target hardware differs from the platform on which the repository was developed. Many machine-learning repositories assume NVIDIA CUDA libraries, build tools, device APIs, and custom kernels;

these components are not directly runnable in a ROCm environment [4, 5]. Simply disabling a CUDA extension or replacing it with a convenient framework fallback may make the program execute while changing the algorithm or performance path being evaluated. Reproducing such work on AMD Instinct™ GPUs therefore requires adapting the software stack while preserving both the paper's scientific protocol and any accelerator-critical implementation.

There is also no single obvious experiment to reproduce. A paper may contain dozens of results spanning models, datasets, seeds, ablations, and hardware scales. Repeating all of them is often impractical, whereas running the smallest example in the repository may say little about the paper's novelty. A useful target must therefore be feasible with the released artifacts while still exercising the methodology behind a central contribution. This choice cannot be made reliably from command visibility or estimated runtime alone.

Current general-purpose coding agents are poorly matched to this objective. Their usual goal is to resolve a software task: make a command complete, repair a build, or satisfy a test [6]. Research reproduction instead asks whether a particular execution constitutes evidence about a published claim. An agent may obtain a clean exit by accepting repository defaults, replacing unavailable data, shrinking the workload, switching to a fallback implementation, or running only a setup check. It may also interpret the launch of a long-running job—or plausible intermediate output—as completion. These actions can be reasonable during debugging, but they can break the connection between the execution and the paper's scientific conclusion.

We introduce **Repo2Reproduce**, an agentic system that reconstructs paper experiments and reproduces their claims on AMD Instinct™ GPUs. Repo2Reproduce begins from the paper, identifies a feasible experiment that tests a stated contribution, reconciles its scientific configuration against the repository, prepares an executable ROCm environment, and runs the complete workflow under controller-owned completion rules. Typed, persisted artifacts connect specialized agents so that the selected target, scientific settings, execution command, and final metric cannot be silently changed between agent's workflow stages.

Specialized reasoning skills combined with controlled tool use distinguish the system. **Bidirectional scientific–configuration reconciliation** maps every paper requirement into the implementation while independently mapping every code-consumed scientific input back to paper evidence. **Execution-failure diagnosis and correction** proposes targeted repairs to failed runs without altering the selected experiment, scientific configuration, metric, or evaluation scope. **Evidence-grounded metric comparison and final reporting** compares the reproduced result with the exact paper target, verifies metric and aggregation semantics, and records deviations, assumptions, and the final verdict. Finally, the **ROCm kernel-porting skill** combines source-analysis and HIPIFY tools with an ordered escalation strategy, numerical-parity checks, and targeted kernel rewrites when direct translation is insufficient. These skills operate through durable state, validated handoffs, synchronous execution, bounded correction, output verification, and evidence-based finalization.

We also introduce **Repo2ReproduceBench**, a benchmark designed specifically to test whether agents can reproduce published claims under a practical compute budget. Unlike software-engineering benchmarks centered on fixing code, passing tests, or resolving repository issues, Repo2ReproduceBench evaluates the complete paper-to-result process: selecting a scientifically meaningful target, reconstructing its protocol, adapting it to AMD Instinct™ GPUs, completing the experiment, and comparing the result with the paper. It comprises a 20-paper claim-level core and a three-task CUDA-kernel porting extension spanning training, checkpoint evaluation, rescoring, certification, profiling, and progressively deeper ROCm adaptation.

Our contributions are:

1. **The Repo2Reproduce agentic system**, which connects paper interpretation, repository analysis, environment preparation, controlled execution, and result finalization through typed, recoverable artifacts.
2. **Bidirectional scientific–configuration reconciliation**, which constructs one evidence-backed ledger from paper-to-code and code-to-paper mappings and gives explicit author-reported settings priority over conflicting generic defaults.
3. **CUDA-to-ROCm porting with GPU-kernel fidelity**, which uses an ordered porting ladder, numerical parity checks, and manifest enforcement to prevent CUDA kernels from being silently replaced by PyTorch fallbacks.
4. **Repo2ReproduceBench**, a 22-task benchmark consisting of a diverse 20-paper reproduction core and a two-task CUDA-kernel porting extension.

2 Related Work

Reproducibility in machine learning. Prior studies show that AI papers frequently omit details needed to reconstruct methods, data, and experiments, motivating reproducibility checklists and artifact-release practices [1, 2, 3]. These efforts primarily improve how authors document experiments. Repo2Reproduce addresses the complementary problem faced by a downstream researcher: recovering a claim-level executable specification when the paper and released repository remain incomplete or inconsistent.

Agents for software and machine-learning research. SWE-bench evaluates agents on resolving repository issues, while MLAgentBench and MLE-bench evaluate iterative experimentation and machine-learning engineering [6, 7, 8]. PaperBench evaluates replication of research papers from scratch, and LMR-BENCH evaluates reconstruction of masked language-modeling code [9, 10]. Two recent systems are especially close to our setting. AutoReproduce mines implicit implementation knowledge from a paper's citation lineage, uses sampling-based unit tests during multi-agent code generation, and introduces a 13-paper benchmark that is also named ReproduceBench [11]. Replica constructs 310 redacted-figure tasks from 100 papers and uses a rubric-based judge to post-train Faraday for one-hour, compute-constrained figure replication [12]. AutoReproduce primarily evaluates generated code against curated reference implementations, whereas Replica permits faithful scaled-down experiments and judges the resulting figure and complete rollout. Repo2Reproduce instead starts from the full paper and its official repository, freezes a contribution-grounded claim and author-supported protocol, preserves the required aggregation, and evaluates execution on ROCm—including CUDA-kernel fidelity—within a practical single-GPU budget.

Research-code reproduction on ROCm. Repo2ROCm is the closest prior system, introducing an agentic workflow for reproducing research repositories on AMD GPUs [13]. Repo2Reproduce extends this direction with branch-specific reproduction ladders, bidirectional scientific–configuration reconciliation, fidelity-preserving kernel porting, controller-owned execution, and a claim-level benchmark.

GPU portability. HIP and HIPIFY support source-level migration of CUDA applications to AMD GPUs, but automated translation may require semantic repairs and architecture-specific optimization [4, 14, 5]. Triton provides a portable language for targeted GPU-kernel rewrites [15]. Repo2Reproduce builds on these tools while requiring numerical parity and explicitly separating functional kernel fidelity from performance agreement.

3 The Repo2Reproduce Agentic System

Repo2Reproduce receives a research paper, the URL of its official code repository, and an optional user-specified target. It produces a sequence of structured artifacts that document the reproduction from selection to final evaluation. The *target contract* identifies the experiment to reproduce, its paper-reported result, metric, and author-specified protocol. The *configuration audit* records the experimental settings required by the paper, their corresponding implementations in the repository, and any conflicts or missing information. The *execution package* specifies the complete environment and procedure needed to run the experiment, including the docker container, dependencies, launch script, expected outputs, and metric-extraction rule. Execution and repair records preserve the logs, failures, modifications, and outcomes of every attempt. Finally, the extracted metric records the reproduced value, while the final report compares it with the paper target and summarizes the configuration, execution status, deviations, evidence, and verdict.

Each run is performed in an isolated directory containing the parsed paper, pinned repository checkout, manifests, logs, patches, and experimental outputs. Persisting these artifacts makes every stage transition inspectable and allows an interrupted run to resume from the first missing or invalid stage. Figure 1 presents the end-to-end architecture and the flow of evidence between stages.

3.1 Target Experiment Selection

The Target Selection Agent reads the entire paper, including appendices, but does not inspect or execute the repository at this stage. If the user requests a target, the agent follows that result exactly. Otherwise, it selects one numerical result representing a central contribution. It records the result, metric, paper value, aggregation, and every author-stated protocol detail including dataset, split, model, checkpoint, seeds, sample count, and evaluation coverage when available.

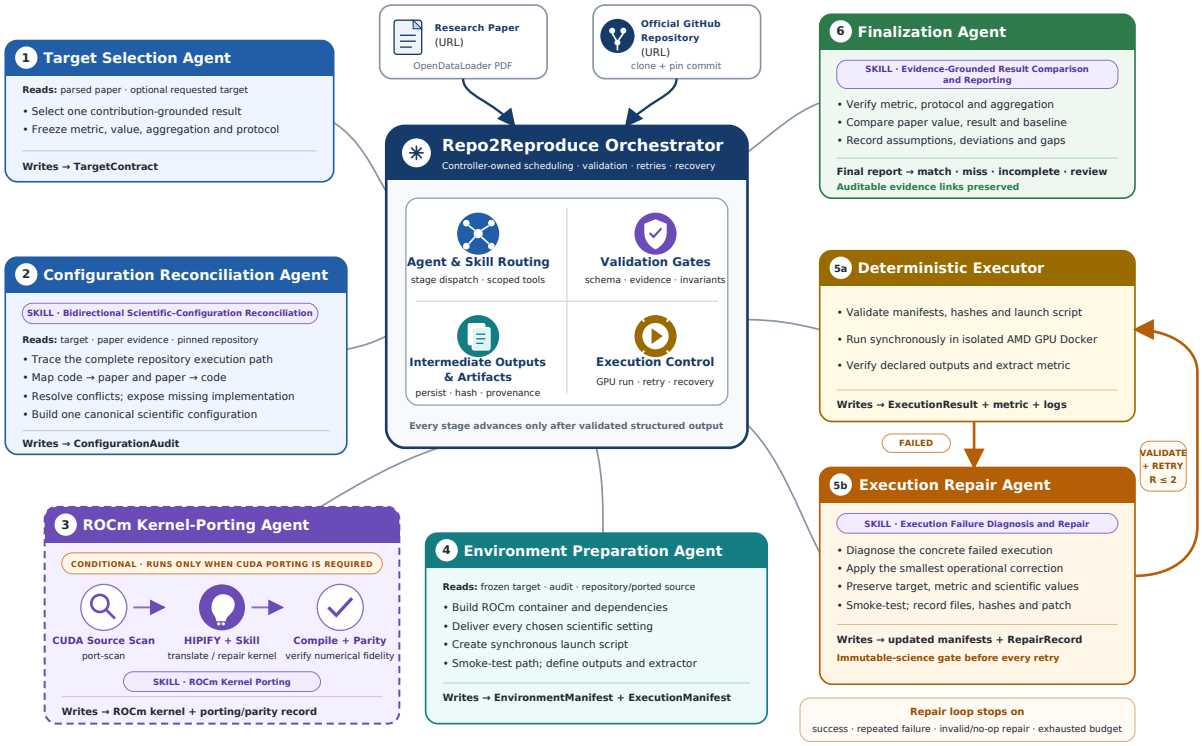


Figure 1: Repo2Reproduce agentic architecture. The paper and official repository are transformed into a frozen target, reconciled scientific configuration, and ROCm execution package. A deterministic controller executes and verifies the experiment, invokes bounded diagnosis and repair after failures, and passes the resulting evidence to finalization. Specialized reasoning skills and stage-specific tools operate over persisted artifacts to preserve the selected experiment and its provenance throughout the workflow.

Each requirement includes a verbatim quotation and paper locator. Missing details remain explicit warnings rather than being filled with repository defaults. The resulting *TargetContract* defines what is to be reproduced.

3.2 Configuration Reconciliation

After the target experiment is frozen, the Configuration Reconciliation Agent applies the **Bidirectional Configuration Reconciliation** skill described in Section 4.1. It maps every author-specified requirement to the selected repository execution path and independently maps every scientific input consumed by the code back to paper evidence. The resulting configuration audit records the paper value, repository value, selected value, supporting evidence, delivery mechanism, and any unresolved conflict for each setting.

3.3 ROCm Kernel Porting

The ROCm Kernel-Porting Agent is invoked only when the selected execution path depends on CUDA-specific components. It applies the porting mechanism described in Section 4.3 to scan the reached CUDA path, select an appropriate migration strategy, and verify numerical parity with a trusted reference. The resulting ROCm implementation and porting evidence are passed to environment preparation without changing the frozen experiment or its scientific configuration.

3.4 Environment Preparation

The Environment Preparation Agent uses the reconciled configuration and, when required, the ported ROCm source to create the complete execution package. This package includes the ROCm container, dependencies, synchronous launch script, bounded smoke test, expected outputs, and metric-extraction rule. Operational details such as package versions and paths may be adapted for ROCm, but the frozen experiment, dataset, scientific settings, metric, sample count, seeds, and aggregation procedure cannot be changed.

3.5 Deterministic Execution and Repair

The Deterministic Executor runs the validated *ExecutionManifest*, which specifies the Docker image, launch script, timeout, expected outputs, scientific configuration, and metric extractor. It executes the experiment synchronously, verifies the declared outputs, and extracts the reproduced metric; partial or detached executions are therefore not treated as completed reproductions.

If execution fails, the Execution Repair Agent applies the failure-diagnosis and repair skill described in Section 4.1. The controller permits a configurable maximum of R repairs while verifying before each retry that the target experiment and resolved scientific configuration remain unchanged. The loop terminates after success, an invalid or repeated repair, or exhaustion of the retry budget.

3.6 Finalization

The Finalization Agent applies the **Evidence-Grounded Result Comparison and Reporting** skill described in Section 4.1. Using the frozen target, execution artifacts, and extracted metric, it compares the reproduced result with the exact paper result and records deviations, ambiguities, and execution status. The final verdict is reported as a match, miss, incomplete run, or blocked target.

4 Skills and Tools

4.1 Specialized Reasoning Skills

Repo2Reproduce equips its agents with three specialized reasoning skills. Each skill is implemented through a stage-specific prompt and structured input–output schema, consumes evidence persisted by preceding stages, and produces an artifact validated by the controller. Unlike tools that perform deterministic operations, these skills define how agents interpret evidence and make stage-specific decisions.

Bidirectional Configuration Reconciliation. This skill checks the selected experiment in two directions. The **code-to-paper pass** traces its actual execution path, identifies every scientifically relevant experiment setting used by the code, and searches the paper for the corresponding value or protocol description. When the paper and repository disagree, the paper value is preferred whenever the implementation supports it. The **paper-to-code pass** starts from every requirement stated by the paper for the selected experiment and verifies that it is implemented and applied in the execution path. Requirements that are missing, only partially implemented, or ignored by the code are explicitly reported. Both passes update a canonical configuration ledger containing the paper and code evidence, resolved value, how that value is applied through a command-line argument, configuration key, checkpoint, or data artifact, and any remaining conflict or omission.

Execution Failure Diagnosis and Repair. This skill is invoked when the deterministic executor reports a failed run. It examines the execution log, environment, repository state, and previous repair attempts to identify the cause of failure. It then proposes a targeted correction to operational problems such as dependency conflicts, build failures, incorrect launch commands, or platform-specific source incompatibilities. Repairs are not allowed to change the selected experiment, dataset, scientific settings, metric, evaluation protocol, or aggregation procedure. The pythn based

deterministic controller validates each proposed correction, records the resulting changes, and decides whether another execution attempt is permitted within the configured retry budget.

Evidence-Grounded Result Comparison and Reporting. This skill compares the reproduced result with the exact paper result defined in the target contract. It verifies that the same metric, dataset, experimental configuration, and aggregation procedure are used before calculating numerical agreement or comparing against a reported baseline. The skill supports individual values as well as vectors, intervals, and multi-run aggregates. Its final report records the paper and reproduced values, comparison method, outcome, configuration deviations, unresolved ambiguities, and supporting execution evidence. This prevents the agent from declaring success using a different result, metric, or experimental setting.

4.2 Stage-Specific Tools

Repo2Reproduce restricts tool access according to each stage's responsibilities. Read, file-discovery, and text-search tools inspect papers, source code, configurations, logs, and results. Write, edit, and shell tools are enabled only during environment preparation and execution repair, where they create containers, launch scripts, configurations, and targeted patches. Experimental commands run inside an isolated GPU-enabled container. Web tools are available during preparation and repair for dependency and compatibility information, but cannot redefine the frozen paper target.

Deterministic utilities outside the LLM parse the paper, clone and pin the repository, compute hashes, validate artifacts, execute the experiment, verify outputs, and extract metrics. This stage-specific access prevents scientific interpretation stages from modifying the experiment or its evidence.

4.3 ROCm Kernel-Porting Mechanism

When the selected experiment depends on CUDA-specific code, the porting agent first determines which components must be adapted to run on AMD GPUs. The `port-scan` tool follows the selected execution path and identifies CUDA kernels, APIs, build rules, and hardware assumptions that may prevent ROCm execution. This report allows the agent to distinguish code that is already portable from code that requires translation or rewriting. When translation is needed, `port-hipify` creates a separate ROCm-oriented copy using AMD's HIPIFY and records what was translated successfully and what still requires correction [14]. These reports provide the evidence used by the agent to choose and apply the next porting step.

Using these reports, the porting agent follows a four-level escalation workflow:

1. **Existing ROCm support:** reuse an official or upstream ROCm implementation.
2. **Portable repository path:** use an equivalent GPU implementation already provided by the repository.
3. **HIPIFY with repairs:** translate the CUDA code and correct the remaining build, API, and hardware-specific incompatibilities.
4. **Targeted kernel rewrite:** rewrite only the non-translatable kernels required by the selected experiment using HIP, Triton, rocWMMA, or Composable Kernel.

The agent stops at the first level that successfully preserves the intended GPU operation. A higher-level framework implementation may be used to verify numerical correctness, but it cannot replace the custom kernel being reproduced.

Compilation is not sufficient evidence of success. The ported operation and a trusted reference are run on identical experiment-relevant inputs under a recorded metric and tolerance. Scan reports, HIPIFY reports, patches, kernel manifests, and parity logs are retained. Numerical parity establishes functional fidelity; it does not imply identical performance across GPU architectures.

5 Repo2ReproduceBench

Repo2ReproduceBench evaluates whether an agent can reproduce a paper-reported result from the paper and its official repository, rather than merely execute the released code. It contains 22 paper–repository tasks: a **20-task core** used for paired comparison with a general-purpose coding agent and a **two-task kernel-porting extension** consisting of NSNQuant and SageAttention [16, 17, 18].

Benchmark construction and diversity. The 20-paper core was curated through a multistage selection process. We first collected papers from major 2025 machine-learning and natural-language-processing venues. The final collection contains seven ICML papers, six ICLR papers, three NeurIPS papers, and four papers from the ACL ecosystem.

We then selected papers to provide diversity in both research domain and experimental workflow. The core contains four NLP papers, four graph-learning papers, four time-series papers, three computer-vision papers, three machine-learning theory or generative-modeling papers, and two efficient-ML or LLM-compression papers. Task diversity was considered independently from domain diversity. The benchmark includes 11 training-centric tasks, three adaptation or post-training tasks, five inference, certification, or theoretical-analysis tasks, and one dataset-and-benchmark task. Collectively, these tasks cover supervised and semi-supervised training, multimodal fine-tuning, unsupervised reconstruction, probabilistic forecasting, continual test-time adaptation, robustness certification, model reprogramming, post-training quantization, prompted LLM evaluation, and generative-model training and sampling.

Finally, we applied practical feasibility constraints. Every selected paper must use a publicly accessible official repository and ungated models, datasets, and evaluation resources; execute locally on one AMD Instinct™ MI355X GPU within four hours; avoid mandatory distributed training or inference. These constraints keep the benchmark computationally manageable while preserving diversity across venues, domains, methods, and execution workflows. The complete list of benchmark papers and their official repositories is provided in Appendix A.

Core Evaluation and Outcomes. Each core task is classified as a **match**, **miss**, or **incomplete**. For a scalar result, the relative error is

$$E_{\mathrm{rel}} = \frac{|m_{\mathrm{run}} - m_{\mathrm{paper}}|}{\max\left(|m_{\mathrm{paper}}|, \epsilon\right)},$$

where m_{run} is the reproduced value, m_{paper} is the paper-reported value, and ϵ prevents division by zero. A completed scalar result is considered a match when

$$E_{\mathrm{rel}} \leq \tau,$$

where τ is the accepted error threshold. We use $\tau = 0.05$ in this study. When the selected claim reports an improvement over a named baseline, satisfying the error threshold alone is insufficient: the reproduced result must also preserve the reported advantage under a comparable dataset, configuration, metric, and evaluation protocol. Vector, interval, structural, and aggregated results use predeclared task-specific agreement rules. A completed comparable result outside its agreement rule is a miss, whereas a failed run, partial protocol, incomplete seed set, or absent comparable output is classified as incomplete.

Kernel-porting extension. The two porting tasks evaluate cases in which the selected contribution depends on a custom CUDA kernel that cannot execute directly on AMD hardware. These tasks are assessed along two separate dimensions. **Kernel fidelity** requires an allowed GPU-kernel implementation, a source-to-port record, and numerical agreement with a trusted reference under a task-specific tolerance. **Performance agreement** compares the locally measured throughput, speedup, or overhead ratio with the corresponding claim reported in the paper. Reporting these dimensions separately prevents a numerically correct but poorly optimized port from being presented as a complete performance reproduction.

Table 1: Claim-level outcomes on the 20-task benchmark core.

System	Match	Miss	Incomplete
General-purpose coding agent	14 (70%)	3 (15%)	3 (15%)
Repo2Reproduce	17 (85%)	3 (15%)	0 (0%)

Table 2: Results on the kernel-porting extension. Numerical fidelity and performance agreement are reported separately.

Task	Strategy	Numerical fidelity	Performance evidence
NSNQuant	HIPIFY with fixes	0.0488% of 8,192 VQ codes differ (2% tolerance)	4.46× throughput and 3× batch; paper reports approximately 3× and 4×
SageAttention	Targeted Triton rewrite	Cosine 0.999921 and relative L1 0.0125 vs. SDPA	0.32× SDPA throughput; paper reports 1.77×

6 Results and Analysis

6.1 Core and Porting Outcomes

Table 1 reports the paired 20-task core. The general-purpose agent matches 14 targets, misses three, and leaves three incomplete. Repo2Reproduce matches 17, misses three, and leaves none incomplete. Its match rate is therefore 85%, compared with 70% for the baseline, while its comparable-result completion rate increases from 85% to 100%.

All three porting tasks execute a GPU-kernel implementation and satisfy their recorded numerical-parity criteria (Table 2). Performance results are mixed. The source papers report NVIDIA measurements whereas the reproductions use an AMD Instinct™ MI355X, so performance ratios are evidence about the current ports, not architecture-independent equivalence claims.

6.2 Qualitative Impact of Skills and Tools

We use representative benchmark cases to examine how the specialized skills and porting mechanism affect reproduction outcomes. For each case, we describe the paper, selected experiment, baseline-agent failure, Repo2Reproduce response, and resulting numerical comparison.

Bidirectional Configuration Reconciliation. **CoGS.** CoGS studies how neural networks compose global solutions through learned algebraic objects [19]. Both agents attempted to reproduce the five-seed $d = 23$ experiment from Table 2, whose paper target is the solution-distribution vector $47.07/11.31/39.80/1.82$ with $\mathrm{err}_6 = 4.80$. The general-purpose agent selected the README learning rate of 0.002, rather than the paper-supported value of 0.01, and later compared stale notebook values instead of its own execution result. It therefore produced no valid paper-comparable number. Repo2Reproduce identified the conflict through bidirectional reconciliation, applied 0.01 to the execution configuration, and completed all five seeds. It reproduced the four distribution values exactly and obtained $\mathrm{err}_6 = 4.82$, a 0.42% difference from the paper target.

Convolutional Diffusion. This paper develops an analytic explanation of creativity in convolutional diffusion models [20]. Both agents attempted to reproduce the Table 1 MNIST experiment, which reports median pixelwise Pearson correlation $r = 0.94$ over 100 samples using 150 network steps. The general-purpose agent evaluated only 50 samples and reported $r^2 = 0.858$, which is not comparable with the paper's unsquared correlation. Repo2Reproduce reconciled the sample count, metric definition, and network-step setting with the paper before execution. Three complete 100-sample runs produced $r = 0.927941 \pm 0.000248$, corresponding to a 1.28% difference from the paper target.

Evidence-Grounded Result Comparison and Reporting. **GrokFormer.** GrokFormer proposes a graph transformer that combines graph Fourier analysis with Kolmogorov–Arnold networks for node classification [21]. The selected experiment is the ten-run Cora result from Table 2, where the paper reports 89.57 accuracy and compares against the HiGCN baseline of 89.23. The reproduced value was 88.8998, only 0.75% below the paper target. The general-purpose evaluation treated this value as a match because it satisfied the numerical error threshold. Repo2Reproduce additionally checked the paper's comparative claim and found that 88.8998 no longer exceeded 89.23. The reporting skill therefore returned a miss, showing that numerical proximity alone does not necessarily reproduce the paper's conclusion.

ROCm Kernel-Porting Mechanism. **NSNQuant.** NSNQuant introduces low-bit vector quantization for reducing KV-cache memory while improving inference throughput [16]. Both systems targeted the comparison between FP16 and NSNQuant-2b, for which the paper reports approximately $3\times$ throughput and a $4\times$ larger batch size. The general-purpose agent did not produce a kernel-faithful ROCm result and therefore had no comparable numerical output. Repo2Reproduce used source scanning, HIPIFY-assisted translation, and targeted repairs to preserve the selected VQ kernel. Only 0.0488% of 8,192 output codes differed from the reference, below the 2% parity tolerance. On the MI355X, the port achieved $4.46\times$ throughput and a $3\times$ larger batch size. It therefore satisfied numerical fidelity and exceeded the reported throughput ratio, but did not reproduce the paper's $4\times$ batch-size ratio.

SageAttention. SageAttention proposes quantized attention kernels for accurate and efficient transformer inference [17, 18]. The selected experiment evaluates the numerical fidelity and reported $1.77\times$ throughput advantage of its INT8 attention kernel. The general-purpose agent could not build the CUDA-specific implementation and produced no numerical or performance result. Repo2Reproduce detected inline PTX and CUDA pipeline operations that prevented direct HIPIFY translation, then rewrote the required kernels in Triton. The port achieved cosine similarity 0.999921 and relative L1 error 0.0125 against SDPA, demonstrating high numerical fidelity. However, its throughput was only $0.32\times$ that of local SDPA, compared with the paper's reported $1.77\times$. This case demonstrates that successful functional porting does not necessarily reproduce architecture-specific performance.

7 Limitations

The 20-task aggregate evaluates the integrated system and does not isolate the causal effect of every skill or tool. The qualitative cases reveal concrete failure modes, but broader component ablations are needed. The benchmark's four-hour, single-GPU constraint excludes distributed and artifact-gated experiments, and some retained core tasks evaluate checkpoints or released outputs rather than complete training pipelines.

The 5% scalar rule is a reporting policy, not a statistical test, and papers differ in variance, seed count, and metric semantics. The three-task porting extension is also too small to characterize CUDA-to-ROCm migration generally. Numerical parity does not ensure that a kernel is performance-optimized, and cross-vendor timing comparisons remain architecture-dependent. Finally, results use one general-purpose-agent baseline; future work should include multiple models, repeated stochastic runs, cost measurements, and blinded human review.

8 Conclusion

Repo2Reproduce treats reproduction as preservation of a scientific contract across paper interpretation, repository analysis, environment preparation, execution, and reporting. Its prompt-based skills reconcile scientific settings, diagnose bounded repairs, and compare results against paper evidence; deterministic tools control mutation, execution, validation, and provenance. A hybrid porting mechanism integrates HIPIFY and source analysis with an escalation policy and numerical-parity checks.

On Repo2ReproduceBench, the system improves the core match rate from 70% to 85% and eliminates incomplete core runs. The porting extension further shows that functional fidelity and performance agreement are distinct outcomes. Together, the system and benchmark provide a compact, auditable basis for evaluating agents that aim to reproduce published results rather than merely execute released code.

References

[1] Odd Erik Gundersen and Sigbjørn Kjensmo. State of the art: Reproducibility in artificial intelligence. *Proceedings of the AAAI Conference on Artificial Intelligence*, 32(1):1644–1651, 2018. doi: 10.1609/aaai.v32i1.11503. URL https://doi.org/10.1609/aaai.v32i1.11503.

[2] Odd Erik Gundersen, Yolanda Gil, and David W. Aha. On reproducible AI: Towards reproducible research, open science, and digital scholarship in AI publications. *AI Magazine*, 39(3):56–68, 2018. doi: 10.1609/aimag.v39i3.2816. URL https://doi.org/10.1609/aimag.v39i3.2816.

[3] Joelle Pineau, Philippe Vincent-Lamarre, Koustuv Sinha, Vincent Larivière, Alina Beygelzimer, Florence d'Alché Buc, Emily Fox, and Hugo Larochelle. Improving reproducibility in machine learning research (a report from the NeurIPS 2019 reproducibility program). *Journal of Machine Learning Research*, 22(164):1–20, 2021. URL https://jmlr.org/papers/v22/20-303.html.

[4] Advanced Micro Devices, Inc. Porting CUDA code to HIP. AMD ROCm HIP documentation, 2026. URL https://rocm.docs.amd.com/projects/HIP/en/latest/how-to/hip_porting_guide.html. Accessed 2026-10-16.

[5] Jun Chen, Xule Zhou, and Hyesoon Kim. CuPBoP-AMD: Extending CUDA to AMD platforms. In *Proceedings of the SC '23 Workshops of the International Conference on High Performance Computing, Network, Storage, and Analysis*, pages 1093–1104. Association for Computing Machinery, 2023. doi: 10.1145/3624062.3624185. URL https://doi.org/10.1145/3624062.3624185.

[6] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=VTF8yNQM66.

[7] Qian Huang, Jian Vora, Percy Liang, and Jure Leskovec. MLAgentBench: Evaluating language agents on machine learning experimentation. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 20271–20309. PMLR, 2024. URL https://proceedings.mlr.press/v235/huang24y.html.

[8] Jun Shern Chan, Neil Chowdhury, Oliver Jaffe, James Aung, Dane Sherburn, Evan Mays, Giulio Starace, Kevin Liu, Leon Maksin, Tejal Patwardhan, Lilian Weng, and Aleksander Mądry. MLE-bench: Evaluating machine learning agents on machine learning engineering. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=6s5uXNWGIh.

[9] Giulio Starace, Oliver Jaffe, Dane Sherburn, James Aung, Jun Shern Chan, Leon Maksin, Rachel Dias, Evan Mays, Benjamin Kinsella, Wyatt Thompson, Johannes Heidecke, Amelia Glaese, and Tejal Patwardhan. PaperBench: Evaluating AI's ability to replicate AI research. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 56843–56873. PMLR, 2025. URL https://proceedings.mlr.press/v267/starace25a.html.

[10] Shuo Yan, Ruochen Li, Ziming Luo, Zimu Wang, Daoyang Li, Liqiang Jing, Kaiyu He, Peilin Wu, Juntong Ni, George Michalopoulos, Yue Zhang, Ziyang Zhang, Mian Zhang, Zhiyu Chen, and Xinya Du. LMR-BENCH: Evaluating LLM agent's ability on reproducing language modeling research. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 6164–6186. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.emnlp-main.314. URL https://aclanthology.org/2025.emnlp-main.314/.

[11] Xuanle Zhao, Zilin Sang, Yuxuan Li, Qi Shi, Weilun Zhao, Shuo Wang, Duzhen Zhang, Xu Han, Zhiyuan Liu, and Maosong Sun. AutoReproduce: Automatic AI experiment reproduction with paper lineage. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2026. URL https://aclanthology.org/2026.acl-long.1001/.

[12] Damon Falck, Samer Sabri, Anja Surina, Thom Foster, Anya Sims, Sam Devlin, Dylan Rogers, Tantum Collins, Kaloyan Aleksiev, Louis Kirsch, and Edward Hughes. Training AI scientists to replicate research. *arXiv preprint arXiv:2608.13331*, 2026. URL https://arxiv.org/abs/2608.13331.

[13] Umang Pandey, Pratik Prabhanjan Brahma, Zicheng Liu, and Emad Barsoum. Repo2ROCm: An agentic system for reproducing research code on AMD ROCm GPUs. In *Proceedings of the 2026 Conference on Empirical Methods in Natural Language Processing: Industry Track*, 2026. Forthcoming.

[14] Advanced Micro Devices, Inc. HIPIFY documentation. AMD ROCm documentation, 2026. URL https://rocm.docs.amd.com/projects/HIPIFY/en/latest/. Accessed 2026-10-16.

[15] Philippe Tillet, H. T. Kung, and David Cox. Triton: An intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, pages 10–19. Association for Computing Machinery, 2019. doi: 10.1145/3315508.3329973. URL https://doi.org/10.1145/3315508.3329973.

[16] Donghyun Son, Euntae Choi, and Sungjoo Yoo. NSNQuant: A double normalization approach for calibration-free low-bit vector quantization of KV cache. In *Advances in Neural Information Processing Systems*, volume 38, pages 48151–48186, 2025. doi: 10.52202/085713-1436. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/3d8ee933c215fbb7b4d1948ff4906299-Abstract-Conference.html.

[17] Jintao Zhang, Jia Wei, Pengle Zhang, Jun Zhu, and Jianfei Chen. SageAttention: Accurate 8-bit attention for plug-and-play inference acceleration. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/b286c344d38e10d2466c0514b78e2f36-Abstract-Conference.html.

[18] Jintao Zhang, Haofeng Huang, Pengle Zhang, Jia Wei, Jun Zhu, and Jianfei Chen. SageAttention2: Efficient attention with thorough outlier smoothing and per-thread INT4 quantization. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 75097–75119. PMLR, 2025. URL https://proceedings.mlr.press/v267/zhang25ae.html.

[19] Yuandong Tian. Composing global solutions to reasoning tasks via algebraic objects in neural nets. In *Advances in Neural Information Processing Systems*, volume 38, 2025. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/666c3cae2f4d0cd796d2113fb797f38d-Abstract-Conference.html.

[20] Mason Kamb and Surya Ganguli. An analytic theory of creativity in convolutional diffusion models. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 28795–28831. PMLR, 2025. URL https://proceedings.mlr.press/v267/kamb25a.html.

[21] Guoguo Ai, Guansong Pang, Hezhe Qiao, Yuan Gao, and Hui Yan. GrokFormer: Graph fourier kolmogorov-arnold transformers. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 21277–21296. PMLR, 2025. URL https://proceedings.mlr.press/v267/ai25d.html.

[22] Giuliano Martinelli, Tommaso Bonomo, Pere-Lluís Huguet Cabot, and Roberto Navigli. BOOKCOREF: Coreference Resolution at Book Scale. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 24526–24544. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.acl-long.1197. URL https://aclanthology.org/2025.acl-long.1197/.

[23] Susanna Rücker and Alan Akbik. Evaluating design decisions for dual encoder-based entity disambiguation. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 15685–15701. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.acl-long.764. URL https://aclanthology.org/2025.acl-long.764/.

[24] Mahalakshmi Sabanayagam, Lukas Gosch, Stephan Günnemann, and Debarghya Ghoshdastidar. Exact certification of (graph) neural networks against label poisoning. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=d9aWa875kj.

[25] Yunju Cho and Jay-Yoon Lee. CoMRes: Semi-supervised time series forecasting utilizing consensus promotion of multi-resolution. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=bRa4JLPzii.

[26] Alessio Pittiglio. Leveraging context for multimodal fallacy classification in political debates. In *Proceedings of the 12th Argument Mining Workshop*, pages 388–397. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.argmining-1.39. URL https://aclanthology.org/2025.argmining-1.39/.

[27] Xingjian Wu, Xiangfei Qiu, Zhengyu Li, Yihang Wang, Jilin Hu, Chenjuan Guo, Hui Xiong, and Bin Yang. CATCH: Channel-aware multivariate time series anomaly detection via frequency patching. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/2b25c39788e5cf11d3541de433ebf4c0-Abstract-Conference.html.

[28] Jiahuan Zhou, Chao Zhu, Zhenyu Cui, Zichen Liu, Xu Zou, and Gang Hua. Class-aware domain knowledge fusion and fission for continual test-time adaptation. In *Advances in Neural Information Processing Systems*, volume 38, pages 62119–62143, 2025. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/50c456f7c4c22fc4f203f94eeb62fc3d-Abstract-Conference.html.

[29] Shuting He, Guangquan Jie, Changshuo Wang, Yun Zhou, Shuming Hu, Guanbin Li, and Henghui Ding. ReferSplat: Referring segmentation in 3d gaussian splatting. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 22456–22467. PMLR, 2025. URL https://proceedings.mlr.press/v267/he25h.html.

[30] Zhichao Hou, MohamadAli Torkamani, Hamid Krim, and Xiaorui Liu. Robustness reprogramming for representation learning. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/84dd970f8f7ebd6986832bdd6445bd5f-Abstract-Conference.html.

[31] Mingyuan Xia, Chunxu Zhang, Zijian Zhang, Hao Miao, Qidong Liu, Yuanshao Zhu, and Bo Yang. TimeEmb: A lightweight static-dynamic disentanglement framework for time series forecasting. In *Advances in Neural Information Processing Systems*, volume 38, 2025. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/6e79ae23fe8a8bb15a2b2972053f4fc3-Abstract-Conference.html.

[32] Chayne Thrash, Reed Andreas, Ali Abbasi, Parsa Nooralinejad, Soroush Abbasi Koohpayegani, Hamed Pirsiavash, and Soheil Kolouri. MCNC: Manifold-constrained reparameterization for neural compression. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/2f63d2963526bdd9ff1b8bcc2dc9905a-Abstract-Conference.html.

[33] Joshua Southern, Yam Eitan, Guy Bar-Shalom, Michael M. Bronstein, Haggai Maron, and Fabrizio Frasca. Balancing efficiency and expressiveness: Subgraph GNNs with walk-based centrality. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 56615–56649. PMLR, 2025. URL https://proceedings.mlr.press/v267/southern25a.html.

[34] Keitaro Sakamoto and Issei Sato. Benign overfitting in token selection of attention mechanism. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 52644–52727. PMLR, 2025. URL https://proceedings.mlr.press/v267/sakamoto25a.html.

[35] Adrien Cortes, Remi Rehm, and Victor Letzelter. Winner-takes-all for multivariate probabilistic time series forecasting. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 11288–11312. PMLR, 2025. URL https://proceedings.mlr.press/v267/cortes25b.html.

[36] Yuxuan Sun, Ruikang Liu, Haoli Bai, Han Bao, Kang Zhao, Yuening Li, Jiaxin Hu, Xianzhi Yu, Lu Hou, Chun Yuan, Xin Jiang, Wulong Liu, and Jun Yao. FlatQuant: Flatness matters for LLM quantization. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 57587–57613. PMLR, 2025. URL https://proceedings.mlr.press/v267/sun25l.html.

[37] Mufei Li, Viraj Shitole, Eli Chien, Changhai Man, Zhaodong Wang, Srinivas Sridharan, Ying Zhang, Tushar Krishna, and Pan Li. LayerDAG: A layerwise autoregressive diffusion model for directed acyclic graph generation. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=pBFURkO06T.

[38] Dayeon Ki, Kevin Duh, and Marine Carpuat. AskQE: Question answering as automatic evaluation for machine translation. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 17478–17515. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.findings-acl.899. URL https://aclanthology.org/2025.findings-acl.899/.

A Repo2ReproduceBench Papers and Repositories

For auditability, we list all papers included in Repo2ReproduceBench together with their publication venues and official code repositories. The benchmark consists of a 20-paper reproduction core and a two-task kernel-porting extension.

A.1 Core Benchmark

- **BOOKCOREF** (ACL 2025): *BOOKCOREF: Coreference Resolution at Book Scale* [22]. Code: link.
- **VerbalizED** (ACL 2025): *Evaluating Design Decisions for Dual Encoder-based Entity Disambiguation* [23]. Code: link.
- **QPCert** (ICLR 2025): *Exact Certification of (Graph) Neural Networks Against Label Poisoning* [24]. Code: link.
- **CoMRes** (ICLR 2025): *Semi-Supervised Time Series Forecasting Utilizing Consensus Promotion of Multi-Resolution* [25]. Code: link.
- **CoGS** (NeurIPS 2025): *Composing Global Solutions to Reasoning Tasks via Algebraic Objects in Neural Nets* [19]. Code: link.
- **MM-ArgFallacy** (ArgMining 2025): *Leveraging Context for Multimodal Fallacy Classification in Political Debates* [26]. Code: link.
- **CATCH** (ICLR 2025): *Channel-Aware Multivariate Time Series Anomaly Detection via Frequency Patching* [27]. Code: link.
- **KFF** (NeurIPS 2025): *Class-aware Domain Knowledge Fusion and Fission for Continual Test-Time Adaptation* [28]. Code: link.
- **Convolutional Diffusion** (ICML 2025): *An Analytic Theory of Creativity in Convolutional Diffusion Models* [20]. Code: link.
- **ReferSplat** (ICML 2025): *Referring Segmentation in 3D Gaussian Splatting* [29]. Code: link.
- **Robustness Reprogramming** (ICLR 2025): *Robustness Reprogramming for Representation Learning* [30]. Code: link.
- **TimeEmb** (NeurIPS 2025): *A Lightweight Static-Dynamic Disentanglement Framework for Time Series Forecasting* [31]. Code: link.
- **MCNC** (ICLR 2025): *Manifold-Constrained Reparameterization for Neural Compression* [32]. Code: link.

- **HyMN** (ICML 2025): *Balancing Efficiency and Expressiveness: Subgraph GNNs with Walk-Based Centrality* [33]. Code: link.
- **Benign Attention** (ICML 2025): *Benign Overfitting in Token Selection of Attention Mechanism* [34]. Code: link.
- **GrokFormer** (ICML 2025): *Graph Fourier Kolmogorov-Arnold Transformers* [21]. Code: link.
- **TimeMCL** (ICML 2025): *Winner-takes-all for Multivariate Probabilistic Time Series Forecasting* [35]. Code: link.
- **FlatQuant** (ICML 2025): *Flatness Matters for LLM Quantization* [36]. Code: link.
- **LayerDAG** (ICLR 2025): *A Layerwise Autoregressive Diffusion Model for Directed Acyclic Graph Generation* [37]. Code: link.
- **AskQE** (Findings of ACL 2025): *Question Answering as Automatic Evaluation for Machine Translation* [38]. Code: link.

A.2 Kernel-Porting Extension

- **NSNQuant** (NeurIPS 2025): *A Double Normalization Approach for Calibration-Free Low-Bit Vector Quantization of KV Cache* [16]. Code: link.
- **SageAttention** (ICLR 2025): *Accurate 8-Bit Attention for Plug-and-play Inference Acceleration* [17, 18]. Code: link.